

What Claude is: models, products, and where it sits versus other LLMs

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 1 of 12 · English notes

Vijay's core definition: **Claude is Anthropic's family of AI models plus the applications that sit on those models.** Anthropic is an AI research and technology company founded in 2021 (the trainer framed it as coming out of OpenAI-era research). The products you click are clients. The models (Haiku, Sonnet, Opus, and others named in class) are the servers. Almost every later feature—Cowork, Code, tools, RAG—only makes sense if you keep that split.

Client-server: products versus models

Chat, Cowork, Claude Code, and apps that call the API are **clients**. They present UI, hold chat history, attach folders, and decide when to call a tool. The LLM is a **stateless server**: it takes a request, returns a response, and does not keep a session on the server. That is why horizontal scaling works and why vendors can put a *zero data retention* posture in the SLA for the model itself.

If Claude Chat “remembers” yesterday’s thread, that memory lives in the *client* (history, project, Cowork memory toggle). On the next turn the client re-sends the context the model needs. Confusing client memory with model memory is a common exam-trap in conversation design.

NAMES IN THE AUGUST 2026 CLASSROOM

Vijay listed Haiku, Sonnet, Opus, and additional family names that appeared in the live picker (including “Fable” / similar ASR). Treat those as *illustrative of the family*. Confirm IDs, context sizes, and which SKUs your org enables on the live [models page](#).

Transformers, generative models, and training cost

The trainer placed all modern Claude-class models on the **transformer** architecture introduced by Google Brain in 2017. They are **generative** models: they produce content (a blog post, a function, a plan), not just a true/false discriminative label. Training is not a weekend notebook job. Vijay’s teaching analogy: training a frontier LLM can cost more in compute than “sending a rocket to the moon”; his GPT-4o-era example was on the order of 100+ million USD plus petabytes of data and large GPU clusters. You will not fine-tune a full LLM on the fly for a UST sprint.

What the weights actually saw

To make “the model already knows programming” concrete, he walked a public Llama-13B-style mix: Common Crawl / CCNet (web pages archived over many years), C4-style consented web text, GitHub (open-source including Kubernetes, Java, Python, .NET), Wikipedia, books, arXiv, Stack Exchange. The point for developers: the model has seen a huge public corpus. It has *not* seen UST’s employee directory or last week’s unpublished LangChain release unless you bring that in via tools, MCP, RAG, or (expensively) custom training.

Cutoff dates, multimodal, and tool calling

After a long training run (he quoted 6–8 months as typical for a frontier cycle), a model is published with a **knowledge cutoff**. In the live Chat demo on 3 August 2026, Claude answered that it had been trained through about January 2026—a six-month gap the model itself flagged. Anything after the cutoff is not “in the weights.” Weather, stock ticks, and last Tuesday’s Confluence page require **tool calling** (function calling): the model plans a call to a REST API, Lambda, Azure Function, web search, or connector; the client executes it; JSON comes back; the model writes the user-facing answer.

- **Multimodal:** Vijay stated that Claude models in this course are multimodal—they can take text, images, and (as discussed in Q&A) other media—not text-only “language models.” Open-source stacks may still ship text-only SLMs/LLMs.
- **Tool calling on all Claude models in class:** that is the feature that makes Chat, Cowork, and Code *agentic* rather than single-shot completion.
- **Do not dump all docs into CLAUDE.md:** a long Q&A warned that stuffing full vendor documentation into markdown is a token killer. Prefer MCP, RAG, or targeted docs search.

How you get data the weights never saw

When SDKs and libraries move faster than the cutoff (the LangChain 0.46 versus 1.2 example), Vijay listed options in order of realism for enterprise:

1. **Do not retrain the LLM daily** for weather or stock data. That is the wrong lever.
2. **Web search / docs search as a tool** — works, burns tokens, still the default for “what is the current API.”
3. **MCP (Model Context Protocol)** — the standard way to plug third-party and latest vendor data into the client; preferred when you may swap Figma for Adobe XD without rewriting the app.
4. **RAG** — your own indexed corpus (version-pinned docs). Pre-populated, not a live query. Good org-wide if the version is standardised after vulnerability review.
5. **Custom / continued training** — expensive; he reserved “train on the fly” language for *small* language models (SageMaker-style), not frontier LLMs.

Inference, tokens, and hallucination (preview)

Because the server is stateless, **inference** means: you send the whole context the model needs, every time, up to a **context window** (classroom numbers: on the order of 200k-class windows, with a 1M option on some Sonnet SKUs—verify live). Tokens are model-specific (he demonstrated a tokenizer; “hi” versus a paragraph is not a fixed character count). Billing is typically input tokens + output tokens; some stacks also bill cached tokens. Claude Chat in the enterprise demo was framed as input/output; prompt cache appears later in the syllabus.

Hallucination in his teaching story: the word “bank” can mean HSBC, a riverbank, or a blood bank if the prompt is underspecified. Fix it with role, task, constraints, and **skills / system instructions** (slash-invoked

in Claude)—knowing that those extra lines *also* consume tokens every turn. Chapter 04 returns to study habits; modules 8–9 return to prompting technique.

Where Claude sits versus “other LLMs”

Vijay compared the family to ChatGPT-style chat and to GitHub Copilot as it evolved from inline assistant to agent. The teaching contrast is not brand loyalty; it is architecture: Claude’s products are clients over Anthropic models with first-class tool use, Cowork as a local agentic workspace, and Claude Code as an agent that can run commands—not a single-file autocomplete. Copilot credit systems and model mix were described as changing rapidly (he mentioned a July 2026 Copilot billing change). Again: exam answers should use Anthropic’s current product language, not a vendor rant.

TRUST DOCS OVER CLASSROOM MODEL IDS

Context sizes (272K vs 1M), effort sliders, and “thinking” / extended-thinking SKUs were demonstrated live. They will not match the exam booklet if Anthropic renamed them. Use **models** and **extended thinking**.

Official documentation

- **Models overview**

<https://docs.claude.com/en/docs/about-claude/models>

Families, context windows, and which capabilities (vision, thinking) attach to which ID.

- **API overview**

<https://docs.claude.com/en/api/overview>

How applications consume the same models the GUI clients use.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Function calling: the mechanism behind web search, connectors, and agents.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Standard connector pattern Vijay contrasted with ad-hoc tools and RAG.

- **Usage policy**

<https://www.anthropic.com/legal/usage-policy>

Enterprise conversations about retention and acceptable use start here.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic’s current docs over session anecdotes. Not official exam material.